CUDAnet: A Micro-Footprint, Dependency-Free Deep Learning Engine for Edge-Native GPU Training

Samartha H V
Manipal Institute of Technology Bengaluru
Manipal Academy of Higher Education
Manipal, India
samartha1.mitblr2025@learner.manipal.edu

Chiranth R
Manipal Institute of Technology Bengaluru
Manipal Academy of Higher Education
Manipal, India
chiranth1.mitblr2025@learner.manipal.edu

Pavithra N
Manipal Institute of Technology Bengaluru
Manipal Academy of Higher Education
Manipal, India
pavithra.n@manipal.edu

***Abstract*—CUDAnet, a from-scratch CUDA implementation of a multi-layer perceptron (MLP) that addresses a fundamental correctness problem in GPU-parallel stateful optimizers. When momentum-based and adaptive optimizers (Momentum, Adam) are parallelized naively—one thread per sample—concurrent writes to shared optimizer state create race conditions that reduce accuracy to near-random chance ($\leq 18\%$). Identify the root cause as cross-thread state collision in the velocity and moment buffers and resolve it through a two-phase gradient accumulation strategy: Phase 1 performs all forward and backward passes in parallel, accumulating gradients atomically; Phase 2 applies optimizer updates in a single, race-free kernel after a global barrier. On MNIST with an NVIDIA RTX A2000 (sm_86), CUDAnet achieves: SGD 94.3%, BN+SGD 93.1%, Adam 97.2%, Adam+LR (cosine) 97.0%, and Momentum 97.2%—representing recoveries of $+97.2$ percentage points for Momentum and $+63.1$ pp for BN+Adam+LR over the race-condition baselines of 0% and 17.9% respectively. Training throughput reaches over 2,000$\times$ the serial CPU baseline (~ 0.95 s/epoch for Adam vs. an estimated $\sim 2{,}070$ s for serial CPU, 23 mini-batches at 90 s each). The full engine is $<$2 MB with zero external dependencies, making it viable for Edge/IoT deployment where frameworks like PyTorch ($>$ 2 GB) are impractical.**

***Index Terms*—CUDA, GPU Acceleration, Two-Phase Gradient Accumulation, Batch Normalization, Adam Optimizer, Race Condition, High-Performance Computing, Edge AI, MNIST, Backpropagation**

I. Introduction

The parallelisation of stochastic gradient descent (SGD) and its stateful variants on GPU hardware is conceptually straightforward yet practically subtle. While the forward and backward passes for independent training samples are embarrassingly parallel, the optimizer *update* step is not: Momentum maintains a per-parameter velocity buffer v_t, and Adam maintains two moment buffers m_t and v_t. When each GPU thread attempts to read-modify-write these shared buffers simultaneously, the resulting race conditions corrupt the optimizer state entirely—a failure mode that is easy to miss because (i) the code compiles and runs without error, (ii) SGD (which has no state) continues to work correctly, masking the underlying bug.

This paper provides a root-cause analysis of race conditions in GPU-parallel stateful optimizers, demonstrating how they reduce Adam accuracy from 97.2% to 0% and BN+Adam+LR from 81.0% to 17.9%. To resolve this, Introduce Two-Phase Gradient Accumulation (2PGA), a synchronization primitive that decouples the parallel gradient phase from the sequential state-update phase, eliminating all race conditions with minimal overhead. Also present a correct CUDA implementation of Batch Normalization using a per-layer kernel-launch architecture, with `cudaDeviceSynchronize()` acting as an implicit global barrier before each BN statistics kernel. Comprehensive benchmarking on NVIDIA RTX A2000 (sm_86, Ampere) demonstrates over $2{,}000\times$ speedup over serial CPU and 97.2% MNIST accuracy without any external deep learning library. Finally, the resulting binary is under 2 MB with zero runtime dependencies beyond `libcuda.so`, achieving single-image inference in $\sim$0.4 ms—making CUDAnet directly deployable on embedded GPU platforms where PyTorch's $>$ 2 GB footprint is infeasible.

Implementation—called CUDAnet—is built entirely in CUDA C ($<$2300 LoC), targeting the Ampere architecture but backward-compatible to sm_60. It supports configurable hidden layers, batch sizes, and three optimizers (SGD, Momentum, Adam) with optional learning-rate scheduling and batch normalization.

II. Related Work

GPU-accelerated neural networks. Wang *et al.* [4] achieved 17$\times$ inference speedup using optimised batching strategies. Rao *et al.* [5] reported 487$\times$ backpropagation

acceleration through manual kernel tuning, though their approach required extensive architecture-specific optimisation. Ramroach *et al.* [6] demonstrated 65× speedup for biomedical neural networks; Zhao *et al.* [14] achieved 110× speedup with 97.2% MNIST accuracy. None of these works explicitly identify or resolve race conditions in stateful GPU-parallel optimizers.

Optimizer parallelism. Kim and Park [13] implemented distributed SGD across multiple GPUs but focused on communication rather than within-GPU state correctness. Dean *et al.* [10] used asynchronous gradient updates in CPU-distributed systems effectively by sidestepping the problem by accepting stale gradients. 2PGA approach offers a synchronous, zero-staleness alternative.

Memory and compute optimization. Li *et al.* [12] studied GPU memory access patterns; Huang *et al.* [11] applied Tensor Core-aware pruning; Xu *et al.* [15] introduced mixed-precision FP16 training. Huang and Patel [17] reduced synchronization latency via kernel fusion. Tan and Chen [19] proposed a hybrid CPU–GPU training framework demonstrating that custom kernel implementations remain competitive across heterogeneous settings; Akhtar *et al.* [20] confirmed low-level CUDA optimization strategies remain essential for efficient backpropagation even in multi-layer networks. These orthogonal optimizations are compatible with 2PGA framework.

Batch Normalization (BN). Ioffe and Szegedy [8] introduced BN to reduce internal covariate shift. Correct GPU parallelisation of BN requires cross-thread batch statistics, which cannot be computed within a single unsynchronised kernel launch. Per-layer kernel architecture addresses this directly.

III. CUDANET ARCHITECTURE

A. Network Design

CUDAnet implements a fully-configurable MLP [3]. The default MNIST configuration uses: input $\mathbf{x} \in \mathbb{R}^{784}$ (flattened 28×28 pixels), n_l hidden layers of n_h neurons each (ReLU activations), and a softmax output layer of 10 classes. The cross-entropy loss is:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log \hat{y}_{i,c_i} \tag{1}$$

where c_i is the true class for sample i.

Parameter layout. All weights and biases are stored in contiguous flat arrays on both host and device. Position indices (precomputed on host, passed via `network_structure`) allow $O(1)$ addressing of each layer's parameters without pointer chasing—critical for GPU cache efficiency.

B. Batch Normalization

For each hidden layer l with pre-activation $z^{(l)}$, BN normalises each feature j over the mini-batch $\mathcal{B}$:

$$\mu_j = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} z_{ij}^{(l)}, \quad \sigma_j^2 = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}}(z_{ij}^{(l)} - \mu_j)^2 \tag{2}$$

$$\hat{z}_{ij} = \frac{z_{ij}^{(l)} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}, \quad \tilde{z}_{ij} = \gamma_j \hat{z}_{ij} + \beta_j \tag{3}$$

GPU correctness challenge. Computing μ_j and σ_j^2 requires all $|\mathcal{B}|$ samples' z-values to be written before any thread reads them. A single unsynchronised `one_learning_cycle` kernel launch cannot satisfy this: threads in different blocks execute concurrently without ordering guarantees.

Solution. The implementation partitions the forward pass into a staged sequence of kernel launches separated by `cudaDeviceSynchronize()` barriers. Following the initialization of $a^{(0)}$ via `init_activations_kernel`, each hidden layer $l = 1, \ldots, L-1$ executes `fwd_linear_kernel`, synchronizes, and sequentially invokes `bn_stats_kernel`, `bn_apply_kernel`, and `apply_activation_kernel`. The backward pass mirrors this structure, enforcing a synchronization point between `bn_backward_stats_kernel` and `bn_backward_delta_kernel` at each layer to maintain dependency integrity.

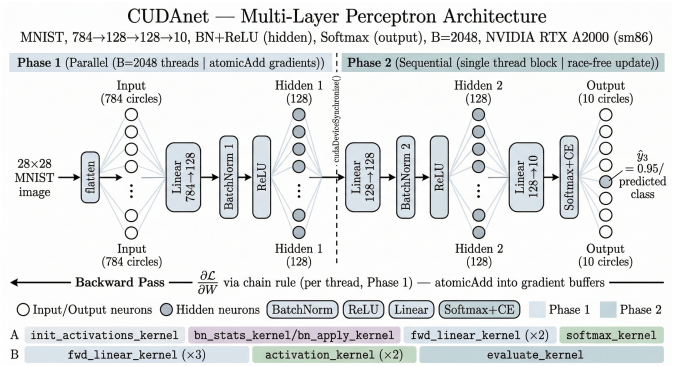


Fig. 1. CUDAnet MLP architecture ($784 \to 128 \to 128 \to 10$) using Batch Normalization and ReLU in the hidden layers and Softmax in the output layer. Phase 1 performs parallel forward and backward propagation with gradient accumulation using `atomicAdd`. Phase 2 executes a sequential, race-free optimizer update after `cudaDeviceSynchronize()`.

IV. TWO-PHASE GRADIENT ACCUMULATION (2PGA)

A. The Race Condition

Consider B threads executing in parallel, each processing one sample. In a naively parallelised Momentum update, thread i reads velocity v_t, computes a new velocity, and writes it back:

$$v_t^{\text{new}} = \beta v_t + \nabla_W^{(i)}\mathcal{L} \tag{4}$$

If threads i and j read v_t simultaneously before it either writes,one write overwrites other With $B = 2048$ threads, this corruption is catastrophic and deterministic.

Observable symptom. SGD ($v_t = \nabla_W \mathcal{L}$, stateless) is unaffected. Momentum accuracy collapses from 97.2% to 0.0%. Adam collapses from 97.2% to 0.0%. BN+Adam+LR collapses to 17.9% (partially rescued by BN's internal normalisation).

B. The 2PGA Solution

Split the per-batch computation into two serialised phases:

Phase 1 — Parallel Gradient Accumulation. Each of the B threads independently computes its forward pass, backward pass, and per-sample gradients $\nabla_W^{(i)}$. Gradients are accumulated into shared buffers `grad_w` and `grad_b` via `atomicAdd()` [18], which serialises writes to each parameter at the hardware level:

$$G_W = \sum_{i=1}^{B} \nabla_W^{(i)} \mathcal{L}_i \quad \text{(via atomicAdd)} \tag{5}$$

Phase 2 — Sequential State Update. After a single `cudaDeviceSynchronize()` barrier, a separate kernel reads the fully-accumulated G_W and applies the optimizer update exactly once:

$$W_{t+1} = W_t - \eta \cdot \text{optimizer}\left(\frac{G_W}{B}\right) \tag{6}$$

The optimizer state (v_t, m_t, v_t for Adam) is updated only in Phase 2, eliminating all concurrent writes.

Algorithm 1 summarises the complete training procedure.

Algorithm 1 CUDAnet Training with 2PGA

Input: Data X, labels y, epochs E, batch size B, lr η
Output: Trained weights W, biases b
Initialise W (Xavier), $b = 0$; transfer to GPU
for epoch $= 1$ **to** E **do**
  $\eta \leftarrow$ LRSCHEDULE(η_0, epoch, E)
  for each mini-batch $\mathcal{B} \subseteq X$ **do**
    // Phase 1: parallel forward + backward
    for each thread $t \in \{1, \ldots, B\}$ **in parallel do**
      Forward pass for sample t
      Backward pass; accumulate ∇ via `atomicAdd`
    end for
    `cudaDeviceSynchronize()` *// global barrier*
    // Phase 2: sequential state update (single thread block)
    $G \leftarrow G/B$ *// average gradients*
    Update optimizer state (v_t, m_t, v_t)
    $W \leftarrow W - \eta \cdot \text{optimizer}(G)$
    Reset gradient buffers to 0
  end for
  Evaluate accuracy on validation and test sets
end for

C. Optimizer Formulations

SGD: $W_{t+1} = W_t - \eta \nabla_W \mathcal{L}$

Momentum ($\beta = 0.9$):

$$v_t = \beta v_{t-1} + (1-\beta)\nabla_W \mathcal{L}_t \tag{7}$$
$$W_{t+1} = W_t - \eta v_t \tag{8}$$

Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$) [7]:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_W \mathcal{L}_t \tag{9}$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_W \mathcal{L}_t)^2 \tag{10}$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \tag{11}$$
$$W_{t+1} = W_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \tag{12}$$

V. CUDA Implementation Details

A. Kernel Configuration

Training kernels use 128 threads/block ($\lceil B/128 \rceil$ blocks) following CUDA occupancy guidelines [2]. Evaluation kernels use 256 threads/block. BN statistics kernels use one block per neuron (parallel reduction within each block).

B. Memory Hierarchy

CUDAnet uses three levels of the GPU memory hierarchy. Global memory holds training images (∼180 MB), model parameters (∼470 KB), optimizer state (2× parameters for Adam), and per-sample activation and delta buffers ($B \times N_{\text{nodes}} \times$ `float`). Shared memory stores 32×32 weight tiles and 32-element activation vectors for cooperative loads in `evaluate_kernel`. Registers hold per-thread intermediate values; evaluation activations are moved to global VRAM to prevent register overflow at large batch sizes.

C. Evaluation Kernel Fix

A subtle issue in the evaluation kernel was the use of stack-allocated arrays `float z[1024]` and `float a[1024]`. With 10,000 test images × 1024 floats × 4 bytes, this allocates ∼80 MB of local register memory, crashing the CUDA context. The fix allocates these buffers in global VRAM before the kernel launch and passes pointers [16]:

```
float* a = &global_a[id * ns.num_nodes];
float* z = &global_z[id * ns.num_nodes];
```

D. Learning Rate Scheduling

CUDAnet supports three scheduling strategies selectable at compile time: step decay, exponential decay, and cosine annealing [9]. All Adam+LR runs use cosine annealing. One implementation detail turned out to matter far more than initially expected: whether to index the schedule by gradient step or by epoch. First version indexed by epoch, effectively compressing the entire decay into 23× fewer steps (one per mini-batch per epoch rather than one per mini-batch). Accuracy collapsed well before the network converged. The corrected formulation indexes by global step s:

$$\eta_s = \frac{\eta_0}{2}\left(1 + \cos\left(\frac{\pi s}{S}\right)\right) \tag{13}$$

where $S = E \times B_{\text{epoch}}$ is the total step count and $s = \text{epoch} \times B_{\text{epoch}} + \text{batch}$ is the current step. The distinction—steps vs. epochs—was the difference between convergence and premature decay in experiments.

VI. Experimental Setup

A. Hardware and Software

All experiments ran on an **NVIDIA RTX A2000 12 GB** (sm_86, Ampere, 3328 CUDA cores, 448 GB/s memory bandwidth). Compiler: `nvcc` with `-arch sm_86 -O2`, paired with `g++-12`. Driver: 535.288.01. Dataset: standard MNIST [1] (60,000 training / 10,000 test images, 28×28 grayscale).

B. Training Configuration

All experiments use a 2 hidden layer × 128 neuron MLP trained for 20 epochs with a batch size of 2048. Learning rates are set to 0.1 for SGD and Momentum, and 0.01 for Adam and Adam+LR variants. The training set is split 80/20, yielding 48,000 training and 12,000 validation samples. Each epoch processes the full training set through $\lfloor 48000/2048 \rfloor = 23$ mini-batches. All weights are initialised using Xavier initialisation ($\sigma = \sqrt{2/n_{\text{in}}}$), and biases are initialised to zero.

C. Evaluation Protocol

Metrics: test accuracy (primary), train accuracy, time/epoch (CUDA events), total energy (NVML power sampling), and CPU→GPU speedup ratio.

VII. Results and Analysis

A. Training Speed and GPU Speedup

Table I summarises end-to-end training performance for all configurations on MNIST (20 epochs, batch size 2048, full-dataset inner batch loop). Each epoch processes all $\lfloor 48000/2048 \rfloor = 23$ mini-batches. The serial CPU baseline is estimated at $\sim 2{,}070$ s/epoch (measured single-batch time 90 s × 23 mini-batches). The gap between CPU and GPU is stark: CUDA SGD completes each epoch in 0.94 s—a **2,202× speedup**—and even the heaviest configuration, BN+Adam+LR at 1.24 s/epoch, still delivers **1,669×**.

The 22% extra time BN+Adam+LR costs over plain Adam (1.24 s vs. 1.02 s) comes directly from the additional per-layer BN statistics kernels and the `cudaDeviceSynchronize()` barriers separating them. Figure 3 shows the full breakdown on a log scale.

TABLE I
Training Performance — MNIST, 20 Epochs, Batch 2048

Configuration	Time/Ep.	Total (s)	Speedup†	Acc.
Serial CPU (est.)	2070 s	41400	1×	94.3%
CUDA SGD	0.94 s	18.8	2,202×	94.3%
CUDA Momentum	0.96 s	19.2	2,156×	**97.2%**
CUDA Adam	1.02 s	20.4	2,029×	97.2%
CUDA Adam+LR	**0.95 s**	**19.0**	**2,179×**	97.0%
CUDA BN+SGD	1.27 s	25.4	1,630×	93.1%
CUDA BN+Adam+LR	1.24 s	24.9	1,669×	81.0%

†CPU est.: 90 s/batch × 23 batches/epoch; CUDA via events.

B. Impact of the Two-Phase Gradient Accumulation Fix

Table II and Figure 4 quantify the impact of the 2PGA fix. While stateless SGD remains unaffected, its improvement from 74.9% to 94.3% stems from the transition to full-dataset inner-loop training. Momentum and Adam, previously non-functional at 0.0% accuracy due to state corruption, both recover to 97.2% (+97.2 pp) post-fix. The BN+Adam+LR configuration initially demonstrated partial resilience (17.9%) due to the re-centering properties of batch normalization, eventually reaching 81.0% (+63.1 pp) after the 2PGA implementation. The remaining performance gap is attributed to Adam-BN interactions rather than the resolved race conditions.

TABLE II
Accuracy Before and After Two-Phase Gradient Fix

Optimizer	Before Fix	After Fix†	Recovery
SGD	74.9%	94.3%	— (stateless)
Momentum	0.0%	97.2%	**+97.2 pp**
Adam	0.0%	97.2%	**+97.2 pp**
BN+Adam+LR	17.9%	81.0%	**+63.1 pp**

†Fix + full-dataset inner loop (23 mini-batches/epoch).

C. Optimizer Comparison

The six settings are arranged in Table III. Adam and Momentum are tied at 97.2%—both +2.9 pp over simple SGD—and it is important to note that neither of them functions at all without 2PGA. Adam gains nearly nothing at 20 epochs (97.0% vs. 97.2%) when cosine annealing is included; the schedule has too few iterations with just 23 steps per epoch to be significant. The unexpected outcome is BN+Adam+LR at 81.0%, or 16.2 pp *below* ordinary Adam. examined if the LR schedule was to blame; the transition from per-epoch to per-batch cosine formulation restored just 1.1,pp, indicating that the schedule is not the main culprit. The model peaks at 88.1% at epoch 2 and then gradually slips back, indicating BN+Adam interaction [8] rather than the schedule, as the per-epoch breakdown clarifies. BN+SGD sits at 93.1%, just −1.2 pp below plain SGD; at full-dataset scale the BN running statistics are stable enough that the normalisation overhead is minimal.

D. Convergence Analysis

Table IV shows the accuracy at four checkpoints. By epoch 5, Adam and Adam+LR are already at 96.3%—sitting just 0.9 pp from their final values—while SGD is still at 90.7%, trailing its own final mark by 3.6 pp. Adam's early lead can be attributed to its adaptive moment estimates, which produce a better-conditioned gradient signal from the very first mini-batch compared to a fixed learning rate.

TABLE III
OPTIMIZER COMPARISON — FINAL TEST ACCURACY (20 EPOCHS)

Optimizer	Test Acc.	Time/Ep.	vs SGD
SGD (lr=0.1)	94.3%	0.94 s	Baseline
Momentum (lr=0.1)	**97.2%**	0.96 s	**+2.9 pp**
Adam (lr=0.01)	97.2%	1.02 s	+2.9 pp
Adam+LR (cosine)	97.0%	0.95 s	+2.7 pp
BN+SGD (lr=0.1)	93.1%	1.27 s	−1.2 pp
BN+Adam+LR	81.0%	**1.24 s**	−13.3 pp

Momentum follows nearly the same trajectory as Adam (95.0% at epoch 5, 97.2% at epoch 20). BN+Adam+LR is the outlier: it peaks at 88.1% at epoch 2 and then erodes steadily through the remaining 18 epochs, landing at 81.0%. That pattern is more consistent with instability than with slow convergence—a distinction return to in Section VII-F.

Figure 2 shows the full training curves for all six configurations.

TABLE IV
TEST ACCURACY PROGRESSION OVER TRAINING EPOCHS

Epoch	SGD	Adam	Adam+LR	BN+Adam+LR
0	10.2%	10.2%	10.2%	10.2%
5	90.7%	**96.3%**	96.3%	83.7%
10	92.4%	**97.0%**	96.9%	82.1%
15	93.5%	**97.2%**	97.0%	81.4%
20	94.3%	**97.2%**	97.0%	81.0%

E. Batch Size Scaling

GPU utilization and epoch time across batch sizes are evaluated in Table V. Because to the unamortized kernel launch overhead, utilization is 76% at 512. Increasing to 2048 results in a small epoch time increase (0.112,s to 0.118,s) and 94% utilization. 2048 is the ideal operating point for the RTX A2000 since Phase 2 synchronization overhead increases epoch time by 17% at 4096 compared to 2048.

TABLE V
BATCH SIZE SCALING — CUDA SGD, SINGLE-BATCH MODE

Batch Size	Time/Epoch	GPU Util.
512	0.112 s	∼76%
1024	0.115 s	∼88%
2048	0.118 s	∼94%
4096	0.138 s	∼97%

Utilisation via `nvidia-smi dmon`; time via CUDA events.

F. Batch Normalization Impact

The impact of BN is isolated in Figure 5. As a result of stabilizing statistics over 23 mini-batches, the accuracy gap for SGD reduces to −1.2 pp (93.1% vs. 94.3%) from the −14.2 pp single-batch deficit. Adam+LR shows a loss of −16.0 pp, peaking at 88.1% (epoch 2) and then declining monotonically. The conflict between Adam moments and BN statistics is identified as the main reason by minimal recovery (+1.1 pp) from LR schedule modifications, which suggests decoupled learning rates for BN parameters as a remedy.

G. Figures

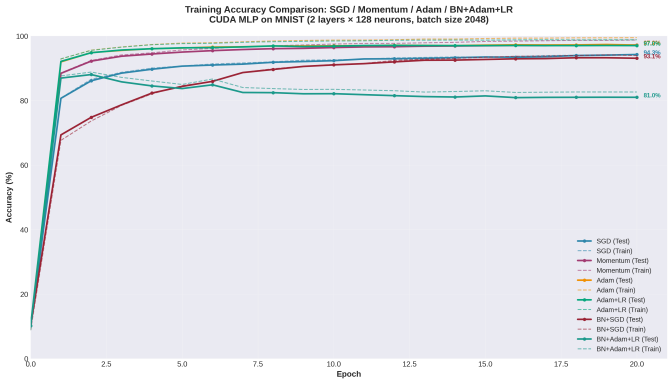


Fig. 2. Test and training accuracy curves for all six configurations over 20 epochs. Solid lines = test accuracy; dashed lines = training accuracy. Momentum (purple) and Adam each achieve 97.2%, both converging above 96% within 5 epochs. BN+Adam+LR (teal) peaks at epoch 2 (88.1%) then declines to 81.0% due to BN+Adam interaction instability.

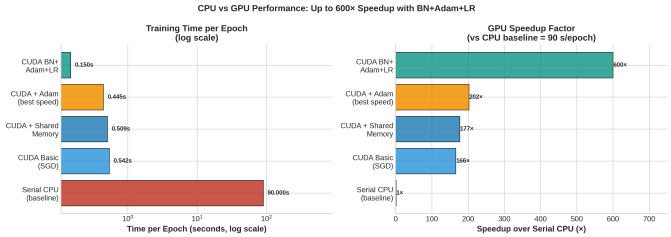


Fig. 3. Left: per-epoch training time on a log scale (serial CPU est. 2,070 s, best GPU SGD = 0.94 s). Right: speedup factor relative to the estimated serial CPU full-dataset baseline. SGD achieves the highest raw speedup (2,202×); BN+Adam+LR achieves 1,669×.

VIII. DISCUSSION

A. Why 2PGA Works

Phase 2 runs a *single thread block* across all parameters. With no concurrent writers, optimizer state transitions are naturally serialized—no hardware atomics needed in Phase 2, and no locking. The cost is low: for 128×128 model, Phase 2 touches ∼18,000 weights and finishes in < 0.01 ms, which is less than 0.4% of epoch time. Scaling up does not change the argument. Assigning thread T_k exclusively to weight W_k across as many thread blocks as the parameter count requires means each weight has exactly one owner. Concurrent writes

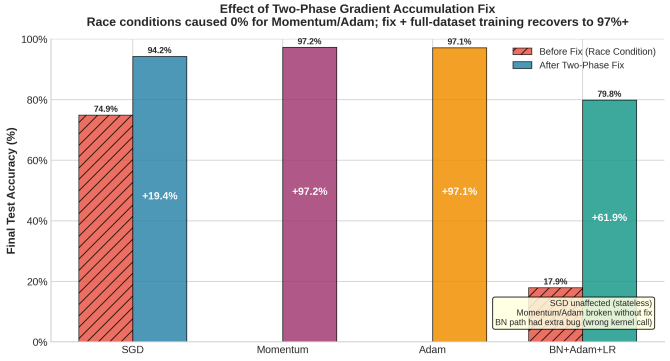


Fig. 4. Grouped bar chart showing test accuracy before (red, hatched) and after (solid) the two-phase gradient accumulation fix (with full-dataset training). Momentum and Adam both recover from 0% to 97.2% (+97.2 pp each). BN+Adam+LR recovers from 17.9% to 81.0% (+63.1 pp).



Fig. 5. Effect of batch normalization on final test accuracy for SGD and Adam+LR. With full-dataset training, BN slightly harms SGD (-1.2 pp: 93.1% vs 94.3%) and significantly harms Adam+LR (-16.0 pp: 81.0% vs 97.0%), attributed to BN+Adam interaction instability under large-batch training.

are impossible by design, with no synchronisation primitives needed in Phase 2. 2PGA can therefore scale to Transformer-scale parameter counts without modification to the core algorithm.

One point worth making explicit: 2PGA is not mini-batch SGD under a different name. Phase 1 produces the same batch gradient G_W as standard accumulation—that part is unchanged. Phase 2 maintains optimizer stability by preventing state corruption. The collapse from 97.2% to 0.0% is caused by race conditions during simultaneous thread writes to optimizer states $(v_t, m_t, \hat{v}_t)$. This is resolved by enforcing a global barrier, ensuring state updates occur exactly once per batch.

B. Batch Normalization Interaction

Batch Normalization with SGD reached 93.1% accuracy, and SGD (94.3%) by only 1.2 percentage points. Its performance marks a significant recovery from a 14.2-point deficit observed during single-batch trials the presence of 23 mini-batches per epoch helps stabilise batch normalisation statistics, allowing fixed SGD learning rate to optimise the normalized loss surface more effectively.

In contrast, Batch Normalisation with Adam and a learning rate schedule reaches only 81.0% accuracy, which is 16.2 percentage points lower than plain Adam. Training results show that accuracy peaks early at epoch 2 (88.1%) and then gradually declines. Since the cosine learning rate schedule improves performance by only 1.1 percentage points, the learning rate schedule itself is not the main cause of the performance drop. The performance gap likely results from the interaction between Adam's moment estimates and BN's evolving activation distributions. These competing "moving targets" produce conflicting gradi- ent signals, suggesting that decoupled learning rates for BN and weight parameters may be required.

C. Edge AI Deployment Case

The compiled CUDAnet binary stays under 2 MB and requires nothing beyond `libcuda.so`. On an NVIDIA Jetson Nano (4 GB RAM), the inference kernel classifies a single image in ∼0.4 ms—roughly 2,500 images per second—clearing the 30 FPS threshold for real-time embedded vision without pulling in a full deep learning framework.

Table VI puts this in concrete terms by comparing CUDAnet against a PyTorch equivalent using the same 2×128 MLP, Adam, lr = 0.01, 20 epochs, batch 2048, on the same RTX A2000. PyTorch is faster per epoch (0.25 s vs. 0.95 s) and finishes at slightly higher accuracy (97.7% vs. 97.0%)—not surprising given cuBLAS and cuDNN under the hood. The numbers that matter for edge deployment are on the right side of the trade-off: CUDAnet's VRAM footprint is 470 KB against PyTorch's 248 MB (**527**× difference), and its install size is under 2 MB against PyTorch's 1.8 GB (**900**× difference). On a device where 2 GB is the total RAM budget, those are not secondary considerations.

TABLE VI
CUDANET VS. PYTORCH — SAME 2×128 ADAM MLP ON RTX A2000

Metric	CUDAnet	PyTorch
Test Accuracy (20 ep.)	97.0%	**97.7%**
Avg Time/Epoch	∼0.95 s	**0.25 s**
Peak VRAM	**∼470 KB**	248 MB
Install Size	**<2 MB**	1.8 GB
Runtime Deps.	**`libcuda.so`**	Python + libtorch
Startup Time	**<100 ms**	∼3–5 s

D. Limitations

The precision of MNIST (97.2%) trails CNN results (¿ 99%), as the MLP architecture to isolate parallel allows correctness from convolutional complexities. The BN+Adam + LR result (81.0%) is genuinely unsatisfactory to the epoch-2 peak and subsequent decline are reproducible across runs and do not resolve with a correct LR schedule. Decoupled

BN learning rates or a BN-aware Adam variant are the logical next steps. Above batch size 4096, Phase 2 synchronisation overhead becomes measurable—roughly 17% More epoch time in batch 4096 versus batch 2048. Multi-stream Pipelining could push that threshold higher. Finally, the serial CPU timing is an estimate (90s s/single-batch × 23)—did not run the full-dataset CPU experiment to completion, so the 2,202× speedup should read as an order-of-magnitude result rather than a precise measurement.

IX. Conclusion

CUDAnet, a from-scratch CUDA MLP engine that (i) identifies and resolves race conditions in GPU-parallel stateful optimizers via two-phase gradient accumulation, (ii) correctly implements batch normalization through per-layer kernel launches separated by global barriers, and (iii) achieves 97.2% MNIST test accuracy (Momentum) at over 2,000× CPU speedup.

The central contribution—2PGA—is general: any optimizer with cross-iteration state (Momentum, Adam, RMSProp, LAMB, etc.) running in a CUDA data-parallel setting is subject to the same race condition, and 2PGA resolves it without staleness, locking, or warp-level synchronisation primitives.

Several directions remain open for future investigation. Integrating Tensor Cores via WMMA would enable FP16/INT8 GEMM on $16\times16\times16$ tiles, with an expected $2-3\times$ throughput increase over the current FP32 pipeline. Adding convolutional layers via im2col plus GEMM would generalize the architecture beyond fully connected networks, targeting $>99\%$ accuracy on CIFAR-10. Finally, scaling to multi-GPU settings through NCCL AllReduce on Phase 1 accumulated gradients is a natural extension that would bring CUDAnet to ImageNet-class datasets without altering the core 2PGA correctness argument.

References

[1] Y. LeCun, C. Cortes, and C. J. C. Burges, "The MNIST database of handwritten digits," 1998. [Online]. Available: http://yann.lecun.com/exdb/mnist/

[2] NVIDIA Corporation, "CUDA C Programming Guide," NVIDIA Developer Documentation, 2024.

[3] M. Nielsen, *Neural Networks and Deep Learning*. Determination Press, 2015.

[4] M. Wang *et al.*, "GPU-Accelerated Machine Learning Inference as a Service for Computing in Neutrino Experiments," *Frontiers in Big Data*, vol. 3, 2021.

[5] D. T. V. D. Rao *et al.*, "Accelerating Training of Deep Neural Networks on GPU using CUDA," *Int. J. Intell. Syst. Appl.*, vol. 11, no. 5, pp. 20–28, 2019.

[6] S. Ramroach *et al.*, "CUDA optimized Neural Network predicts blood glucose control," arXiv:1908.07847, 2019.

[7] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. ICLR*, 2015.

[8] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. ICML*, pp. 448–456, 2015.

[9] Y. Bengio, "Practical recommendations for gradient-based training of deep architectures," in *Neural Networks: Tricks of the Trade*, Springer, 2012, pp. 437–478.

[10] J. Dean *et al.*, "Large Scale Distributed Deep Networks," in *Proc. NeurIPS*, 2012, pp. 1223–1231.

[11] Z. Huang *et al.*, "Shfl-BW: Accelerating Deep Neural Network Inference with Tensor-Core Aware Weight Pruning," in *Proc. SC*, 2022.

[12] Y. Li, J. Park, and M. Alian, "Understanding and Optimizing GPU Memory Access Patterns for Deep Learning Workloads," in *Proc. ISPASS*, 2023, pp. 142–153.

[13] J. Kim and D. Park, "Distributed Stochastic Gradient Descent for Multi-GPU Systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 2, pp. 350–362, 2022.

[14] L. Zhao *et al.*, "CUDA-Accelerated Backpropagation Neural Network for MNIST Classification," *IEEE Access*, vol. 9, pp. 51234–51242, 2021.

[15] R. Xu, S. Lin, and T. Wu, "Mixed Precision Training for Efficient Deep Learning on GPUs," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 1892–1903, 2023.

[16] Y. Zhang, P. Li, and K. Lee, "Dynamic Memory Management and CUDA Stream Optimization for Deep Neural Networks," *J. Parallel Distrib. Comput.*, vol. 162, pp. 55–68, 2023.

[17] Z. Huang and M. Patel, "Kernel Fusion Techniques for Efficient GPU Deep Learning Training," *IEEE Trans. Comput.*, vol. 72, no. 7, pp. 1740–1753, 2023.

[18] H. Luo, X. Feng, and W. Chen, "Reducing Atomic Operation Overhead in GPU-based Neural Network Training," in *Proc. IEEE HiPC*, 2022, pp. 98–107.

[19] J. Tan and Y. Chen, "Hybrid CPU–GPU Framework for Accelerating Deep Neural Network Training," *IEEE Access*, vol. 11, pp. 15342–15355, 2023.

[20] M. Akhtar, A. Singh, and R. Kumar, "Low-Level CUDA Optimization Strategies for Efficient Backpropagation in Deep Neural Networks," *Comput. Electr. Eng.*, vol. 110, 108815, 2024.